SE Lab 5 - Reflections

Name: Pranav Prabhakar

SRN: PES1UG24AM463

Class: 5-H

Topic: 43 - Hotel Booking System

1. Which issues were the easiest to fix, and which were the hardest? Why?

The easiest fixes were mechanical style issues: the unused `math` import, the missing final newline, `== None` changed to `is None`, the camelCase function name and the missing docstrings. Each tool names the exact line and the fix is obvious.

The hardest was the `eval()` call in `compute_seasonal_rate`. Deleting it was not enough, because the function's whole interface depended on receiving an expression string. I had to change its parameter to a numeric multiplier and update the caller, while keeping the same output (187.5). The mutable default argument also took more thought. The fix itself is short, but the shared `[]` meant the booking log was never visible, so I also had to change `main()` to pass in its own log and print it.

2. Did the static analysis tools report any false positives? If so, describe one example.

Some findings depend on context:

- **Bandit B311** (insecure random) on `generate_booking_ref()` is arguably a false positive. A booking reference is not a cryptographic secret, so `random` would be fine for many hotels. I still switched to `secrets`, because guessable references could let someone look up another guest's booking.
- **Bandit B324** (MD5) depends on whether the hash is used for security. Hashing a guest ID only as a lookup key could be marked with `usedforsecurity=False`. Since the purpose was unclear, SHA-256 was the safer choice.
- **Pylint W0612** (unused variable `base_price`) was misleading rather than false. The variable was used, but only inside the string passed to `eval()`, which Pylint cannot see. In this case the warning helped expose the `eval` problem.

3. How would you integrate static analysis tools into your actual software development workflow?

- **Locally:** run Flake8 and Pylint in the editor, and add a `pre-commit` hook that runs Flake8, Pylint and Bandit so problems are caught before each commit.
- **In CI:** run all three tools on every pull request (for example with GitHub Actions). The build fails on any Bandit High or Medium finding, any Flake8 error, or a Pylint score below a set threshold such as 9.5.

- **Configuration:** keep shared settings in `pyproject.toml` or `.flake8` so everyone uses the same rules. When a finding is a confirmed false positive, suppress it on that line (`# nosec`, `# pylint: disable=...`) with a comment explaining why, instead of turning the check off everywhere.

4. What tangible improvements did you observe in code quality, readability, or robustness after applying the fixes?

- **Security:** the shell injection, arbitrary code execution through `eval`, the MD5 hash and the hardcoded, printed secret are all gone. Bandit went from 6 issues (2 High) to 0.
- **Correctness:** bookings no longer share a hidden log between calls. `cancel_booking` now reports unknown rooms instead of hiding the error. The ledger file is always closed, even if writing fails. Importing the module no longer runs the demo.
- **Readability:** every function has a docstring, names follow PEP 8, and `check_room_status` is now one clear line instead of an if/else chain.
- **Measurable:** Pylint went from 5.32/10 to 10.00/10 and Flake8 from 7 issues to 0. The program's output stayed the same apart from the intended changes.